

# Operating Systems and Concurrency

Processes 2, Scheduling

COMP2007

Dan Marsden

(Chao Chen)

{Chao.Chen, Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham

United Kingdom

2026

# Recap

## Last Lecture

- Processes have **control structures** associated with them, the **process control blocks** and **process tables**.
- Processes can have different **states** and the kernel triggers **transitions** between these states.
- The operating system maintains multiple **process queues**.
- The operating system **manages processes** on the user's behalf.

# Goals for Today

## Overview

- Introduction to process scheduling
- Types of **process schedulers**
- **Evaluation criteria** for scheduling algorithms
- Typical **process scheduling algorithms**

# Process Scheduling

## Context

- The OS is responsible for **managing** and **scheduling processes**
  - Decide when to **admit** processes to the system (new → ready)
  - Decide which process to **run** next (ready → run)
  - Decide when and which processes to **interrupt** (running → ready)
- It relies on the **scheduler** (dispatcher) to decide which process to run next, which uses a **scheduling algorithm** to do so
- The type of algorithm used by the scheduler is influenced by the type of operating system (e.g., real time vs. batch)

# Process Schedulers

## Classification by Time Horizon

- **Long term:** applies to new processes and controls the degree of multiprogramming by deciding which processes to admit to the system when
  - A good mix of CPU and I/O bound processes is favourable to keep all resources as busy as possible
  - Usually absent in popular modern OS
- **Medium term:** controls swapping and the degree of multi-programming
- **Short term:** decide which process to run next
  - Manages the ready queue
  - Invoked very **frequently**, hence must be **fast**
  - Usually called in response to **clock interrupts**, **I/O interrupts**, or **blocking system calls**

# Process Schedulers

## Classification by Time Horizon

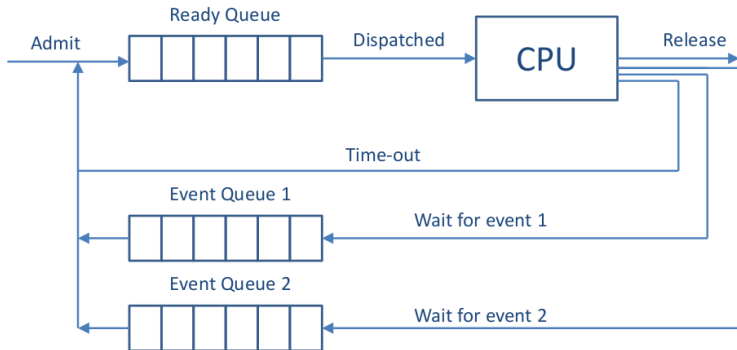


Figure: Queues in OS

# Process Schedulers

## Classification by Approach

- **Non-preemptive**: <sup>非抢占式的</sup> processes are only interrupted <sup>voluntary</sup> voluntarily (e.g., I/O operation or “nice” system call – `yield()`)
  - Windows 3.1 and DOS were non-preemptive
- **Preemptive**: processes can be interrupted forcefully or voluntarily
  - Typically **driven by interrupts from a system clock**.
  - Requires additional context switches which generate **overhead**, too many of them should be avoided (recall last lecture)
  - Prevents processes from monopolising the CPU <sup>垄断</sup>
  - Most popular modern operating systems are preemptive

# Performance Assessment

## Criteria

- **User oriented criteria:**

- **Response time:** minimise the time between creating the job and its first execution
- **Turnaround time:** minimise the time between job creation and completion
- **Predictability:** minimise the variance in processing times

- **System oriented criteria:**

- **Throughput:** maximise the number of jobs processed per hour
- **Fairness:**
  - Are processing power/waiting time equally distributed?
  - Are some processes kept waiting excessively long (starvation)

- Evaluation criteria can be **conflicting**, i.e., **improving the response time** may require **more context switches**, and hence **worsen the throughput** and **increase the turn around time**

# Scheduling Algorithms

## Overview

- **Algorithms** considered:

- ① First Come First Served (FCFS)/ First In First Out (FIFO)
- ② **Shortest job first**
- ③ **Round robin** 轮转调度
- ④ **Priority queues**

- Performance measures used:

- **Average response time:** the average of the time taken for all the processes to start
- **Average turnaround time:** the average time taken for all the processes to finish

# Scheduling Algorithms

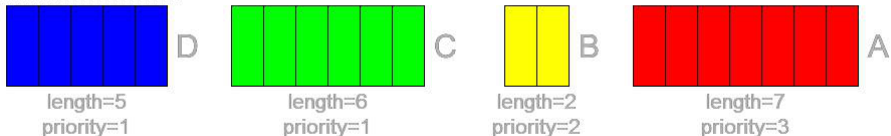
## First Come First Served

- Concept: a **non-preemptive algorithm** that operates as a **strict queueing mechanism** and schedules the processes in the same order that they were added to the queue
- Advantages: **positional fairness** and easy to implement
- Disadvantages:
  - **Favours long processes** over short ones (think of the supermarket checkout!)
  - Could **compromise resource utilisation**, i.e., CPU vs. I/O devices

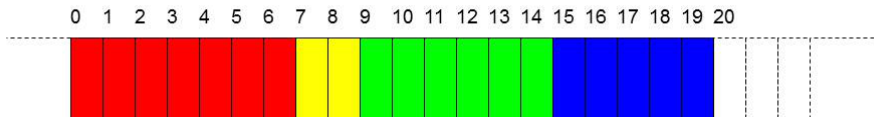
# Scheduling Algorithms

## First Come First Served

### Process Queue



### CPU



- Average response time =  $\frac{0+7+9+15}{4} = \frac{31}{4} = 7.75$
- Average turn around time =  $\frac{7+9+15+20}{4} = \frac{51}{4} = 12.75$

# Scheduling Algorithms

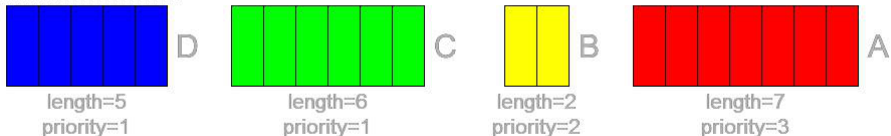
## Shortest Job First

- Concept: A **non-preemptive algorithm** that starts processes in order of **ascending processing time** using a provided/known estimate of the processing
- Advantages: always result in the **optimal turn around time**
- Disadvantages:
  - **Starvation** might occur
  - **Fairness** and **predictability** are compromised
  - **Processing times have to be known** beforehand

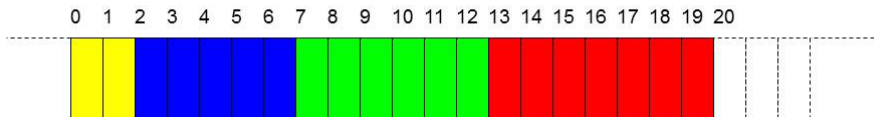
# Scheduling Algorithms

## Shortest Job First

### Process Queue



### CPU



- Average response time =  $\frac{0+2+7+13}{4} = \frac{22}{4} = 5.5$
- Average turn around time =  $\frac{2+7+13+20}{4} = \frac{42}{4} = 10.5$

# Scheduling Algorithms

## Shortest Job First

Why does shortest job first give optimal turn around times?

# Scheduling Algorithms

## Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes,  $P_1$ ,  $P_2$ ,  $P_3$  and  $P_4$ .

# Scheduling Algorithms

## Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes,  $P_1, P_2, P_3$  and  $P_4$ .
- Assume the processes are run in order, and the  $i^{th}$  process requires  $T_i$  milliseconds.

# Scheduling Algorithms

## Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes,  $P_1, P_2, P_3$  and  $P_4$ .
- Assume the processes are run in order, and the  $i^{th}$  process requires  $T_i$  milliseconds.
- Let  $S_i$  be the turnaround time of the  $i^{th}$  process.

# Scheduling Algorithms

## Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes,  $P_1, P_2, P_3$  and  $P_4$ .
- Assume the processes are run in order, and the  $i^{th}$  process requires  $T_i$  milliseconds.
- Let  $S_i$  be the turnaround time of the  $i^{th}$  process.
- Then:

$$S_1 = T_1 \quad S_2 = T_1 + T_2 \quad S_3 = T_1 + T_2 + T_3 \quad S_4 = T_1 + T_2 + T_3 + T_4$$

# Scheduling Algorithms

## Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes,  $P_1, P_2, P_3$  and  $P_4$ .
- Assume the processes are run in order, and the  $i^{th}$  process requires  $T_i$  milliseconds.
- Let  $S_i$  be the turnaround time of the  $i^{th}$  process.
- Then:

$$S_1 = T_1 \quad S_2 = T_1 + T_2 \quad S_3 = T_1 + T_2 + T_3 \quad S_4 = T_1 + T_2 + T_3 + T_4$$

- Therefore the average turnaround time is:

$$\frac{4 \times T_1 + 3 \times T_2 + 2 \times T_3 + 1 \times T_4}{4}$$

# Scheduling Algorithms

## Round Robin

- Concept: a preemptive version of FCFS that forces context switches at periodic intervals or time slices
  - Processes **run in the order that they were added** to the queue
  - Processes are forcefully **interrupted by the timer**
- Advantages:
  - Improved **response time**
  - Effective for general purpose interactive/time sharing systems
- Disadvantages:
  - Increased context switching and thus overhead
  - Favours CPU bound processes (which usually run long) over I/O processes (which do not run long)
    - Can be prevented by working with multiple queues?
  - Can reduce to FCFS      Yes: Multilevel Queue / Multilevel Feedback Queue

---

Exam 2013-2014: Round Robin is said to favour CPU bound processes over I/O bound processes. Explain why may this be the case (if this is the case at all)?

# Scheduling Algorithms

## Round Robin

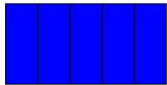
Exam 2013-2014: Round Robin may favour CPU-bound processes because they tend to consume their full time quantum, whereas I/O-bound processes frequently block before the quantum expires. When an I/O-bound process becomes ready again, it is typically placed at the end of the ready queue and must wait again, while CPU-bound processes continue to receive full quanta. This can result in poorer response for I/O-bound processes.

- The **length** of the **time slice** must be carefully considered!
- For instance, assuming a **multi-programming system** with **preemptive scheduling** and a **context switch time** of 1ms:
  - E.g., a **good (low) response time** is achieved with a **small time slice** (e.g. 1ms)  $\Rightarrow$  low throughput
  - E.g., a **high throughput** is achieved with a **large time slice** (e.g. 1000ms)  $\Rightarrow$  high response time
- If a time slice is only **used partially**, the next process **starts immediately**

# Scheduling Algorithms

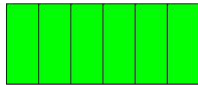
## Round Robin

### Process Queue



length=5  
priority=1

D



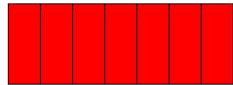
length=6  
priority=1

C



length=2  
priority=2

B

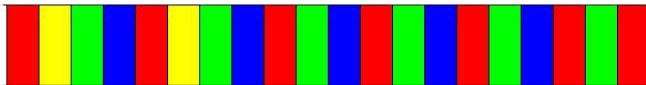


length=7  
priority=3

A

### CPU

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20



- Average response time =  $\frac{0+1+2+3}{4} = \frac{6}{4} = 1.5$
- Average turn around time =  $\frac{6+17+19+20}{4} = \frac{62}{4} = 15.5$

# Scheduling Algorithms

## Priority Queues

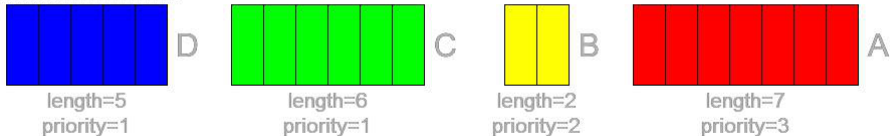
- Concept: A **preemptive algorithm** that schedules processes by priority (high  $\rightarrow$  low)
  - A **round robin** is used **within the same priority levels**
  - The process priority is saved in the **process control block**
- Advantages: can **prioritise I/O bound jobs**
- Disadvantages: low priority processes may suffer from **starvation** (when priorities are static)

Shortest Job First and Highest Priority First can lead to starvation. In SJF, a long process may wait indefinitely if shorter processes keep arriving. In priority scheduling, a low-priority process may wait indefinitely if higher-priority processes continue to arrive. FCFS and Round Robin normally do not cause starvation because processes are eventually served in arrival or cyclic order.

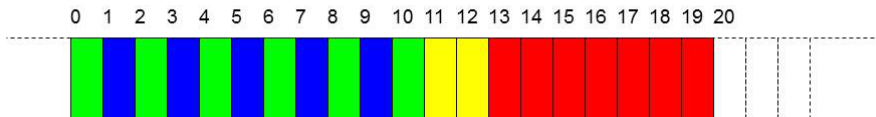
# Scheduling Algorithms

## Priority Queues

### Process Queue



### CPU



- Average response time =  $\frac{0+1+11+13}{4} = \frac{25}{4} = 6.25$
- Average turn around time =  $\frac{10+11+13+20}{4} = \frac{54}{4} = 13.5$

# Scheduling Algorithms

## Priority Queues

- Give the **order in which the processes are scheduled** when using **priority queues**, together with the **times at which they will start, end, and are interrupted** (all processes are available at the time of scheduling)
- You can assume a time slice of 15 milliseconds
- Calculate the **average response and turn around time**

|                  | FCFS Position | CPU burst time | Priority |
|------------------|---------------|----------------|----------|
| <b>Process A</b> | 1             | 67             | 1 (high) |
| <b>Process B</b> | 2             | 37             | 1 (high) |
| <b>Process C</b> | 3             | 14             | 2 (low)  |
| <b>Process D</b> | 4             | 16             | 2 (low)  |

# Scheduling Algorithms

## Priority Queues

- Solution:

- Sequence: A(15)  $\Rightarrow$  B(15)  $\Rightarrow$  <sup>Important</sup> A(15)  $\Rightarrow$  B(15)  $\Rightarrow$  A(15)  $\Rightarrow$  B(7)  $\Rightarrow$  A(15)  $\Rightarrow$  A(7)  $\Rightarrow$  C(14)  $\Rightarrow$  D(15)  $\Rightarrow$  D(1)
- Average response time =  $(0 + 15 + 104 + 118) / 4$
- Average turnaround time =  $(82 + 104 + 118 + 134) / 4$

- Note: we ignore context switch time

# Test your understanding

1. A process can continue performing ordinary CPU computations without giving up the CPU, provided it does not perform blocking I/O or voluntarily yield, including arithmetic calculation, loop, memory computation, normal instructions

- On a non-preemptive operating systems, what sort of things can a process do without potentially “volunteering” to <sup>give up</sup> cede the CPU to another process.
- Using the non-preemptive shortest job first scheduler, does the shortest job run on the CPU until it is completed?
- On a non-preemptive operating system, how much harm can a single user process do?

2. No. Non-preemptive means the process cannot be forcibly preempted, but it may still voluntarily give up the CPU, for example by blocking on I/O.

3. A single user process can potentially monopolise the CPU. If it never blocks, terminates, or voluntarily yields, the operating system cannot preempt it, so other processes may be prevented from running indefinitely.

# Processes 2: Scheduling – Core Revision Summary

process queues · schedulers · preemption · performance criteria · FCFS · SJF · Round Robin · priority queues ♦ source-marked / dialogue-emphasised content

## 1. Topic Overview

This lecture explains how the OS decides **which process gets the CPU, when it gets it, and when it must give it up**. It links process-state transitions and queues to scheduler types, then compares four scheduling algorithms using response time, turnaround time, fairness and throughput. The central trade-off is **responsiveness vs. scheduling overhead/fairness**.

**Lecture logic:** process states / queues → scheduler time horizons → preemptive vs. non-preemptive → evaluation criteria → FCFS / SJF / Round Robin / priority queues.

## 2. Core Concepts

### Process scheduling and state transitions

- ▶ The OS decides when to **admit** a process (new → ready), which ready process to **dispatch** (ready → running), and when to interrupt a running process (running → ready). The scheduler/dispatcher applies a scheduling algorithm. (pp.4–6)
- ▶ ♦ **Ready queue** = processes able to run and waiting only for CPU time. **Event queue** = processes blocked waiting for a particular event/I/O completion. When the event occurs, the process normally returns to the ready queue, *not directly to the CPU*.
- ▶ ♦ **Time-out** means the running process has exhausted its time slice: running → ready. It is a normal scheduling event, *not* a program error/timeout failure.

### Schedulers by time horizon

- ▶ **Long-term scheduler:** decides which new processes are admitted; controls degree of multiprogramming and ideally keeps a useful mix of CPU-bound and I/O-bound jobs. Often absent in modern general-purpose OSs. (p.5)
- ▶ **Medium-term scheduler:** controls swapping and therefore the degree of multiprogramming. (p.5)
- ▶ **Short-term scheduler:** selects the next process from the ready queue. It runs very frequently and therefore must be fast; commonly triggered by clock interrupts, I/O interrupts or blocking system calls. (p.5)

### Non-preemptive vs. preemptive

- ▶ **Non-preemptive:** a process gives up the CPU only **voluntarily**, e.g. blocking for I/O or calling `yield()`. The OS does not forcibly remove it just because another process is waiting. (p.7)
- ▶ ♦ Non-preemptive does **not** mean “must run to completion”: a process can still block for I/O or voluntarily yield before finishing.
- ▶ **Preemptive:** a process may be interrupted voluntarily or **forcefully**, usually through periodic system-clock/timer interrupts. This prevents a process monopolising the CPU, but causes more context switches. (p.7)
- ▶ ♦ **Context-switch overhead:** CPU time/resources spent saving the old process state and restoring the next process state. It is necessary management work, but does not directly advance user computation; too many switches reduce efficiency.

**Non-preemptive danger:** a badly behaved user process that never terminates, blocks or yields can monopolise the CPU and make other processes wait indefinitely, potentially making the system appear unresponsive.

### Performance criteria

- ▶ **Response time:** time from job creation/arrival until its **first execution**. **Turnaround time:** time from job creation/arrival until **completion**. (pp.8–9)
- ▶ **Predictability:** minimise variation in processing times. **Throughput:** maximise jobs completed per unit time. **Fairness:** avoid excessive waiting / starvation. (p.8)
- ▶ Criteria conflict: shorter response time can require more context switches, increasing overhead and potentially lowering throughput / worsening turnaround time. (p.8)

# Processes 2: Scheduling – Core Revision Summary

## 3. Scheduling Algorithms

### 1) First Come First Served (FCFS / FIFO)

► **Non-preemptive**; processes run in the order they entered the queue. (p.10) ► Advantages: simple, positional fairness. Disadvantages: long jobs can delay short jobs; resource utilisation can suffer when CPU/I/O-bound jobs are poorly mixed. ► **Starvation?** Normally no: a process may wait a long time, but its queue position eventually reaches the front.

### 2) Shortest Job First (SJF)

► **Non-preemptive**; selects jobs in ascending estimated CPU processing time. It requires processing-time estimates in advance. (p.12) ► Advantage: gives the **optimal average turnaround time** under the lecture assumptions. Disadvantages: fairness/predictability suffer; long jobs can starve. (p.12) ► ♦ **Why optimal?** With four jobs in order  $T_1, T_2, T_3, T_4$ , total turnaround contribution is  $4T_1 + 3T_2 + 2T_3 + T_4$ ; therefore the shortest duration should receive the largest coefficient by being first. (pp.18–19)

### 3) Round Robin (RR)

► **Preemptive version of FCFS**: processes keep FIFO order, but a timer forces a context switch after a fixed **time quantum / time slice**. (p.20) ► If a process finishes or blocks before using its whole quantum, the next process begins immediately. (p.21) ► Advantages: improved response time; effective for interactive/time-sharing systems. Disadvantages: more context-switch overhead; quantum selection is critical. (pp.20–21)

**Quantum trade-off**: small quantum  $\Rightarrow$  fast response but many switches and lower throughput. Large quantum  $\Rightarrow$  fewer switches / higher throughput but poorer response. As the quantum becomes very large, RR **reduces toward FCFS**.

► ♦ **Why RR may favour CPU-bound jobs**: CPU-bound processes usually consume a full quantum, while I/O-bound processes often run briefly, block for I/O, and later rejoin the ready queue. They may repeatedly wait behind CPU-bound jobs after each I/O completion. Multiple/priority queues can compensate by favouring interactive/I/O-bound jobs.

### 4) Priority Queues / Highest Priority First

► **Preemptive**; choose the highest-priority ready process. Within the same priority level, the lecture uses Round Robin. Priority is stored in the PCB. (p.23) ► Advantage: can prioritise I/O-bound / interactive jobs. Disadvantage: with static priorities, low-priority processes may suffer **starvation**. (p.23)

## 4. Key Formulae / Rules

For process  $i$  with arrival time  $A_i$ , first CPU start time  $F_i$ , and completion time  $C_i$ :

$$\begin{aligned} \text{Response time}_i &= F_i - A_i, & \text{Turnaround time}_i &= C_i - A_i \\ \text{Average response} &= \frac{\sum_i (F_i - A_i)}{n}, & \text{Average turnaround} &= \frac{\sum_i (C_i - A_i)}{n} \end{aligned}$$

In the lecture's worked examples all jobs are available at scheduling time, so  $A_i = 0$  and the values reduce to start/completion times.

### Priority-queue worked pattern (pp.25–26)

With A/B at high priority, C/D at low priority and quantum = 15 ms, the high-priority queue completes first using RR:

$$A(15) \rightarrow B(15) \rightarrow A(15) \rightarrow B(15) \rightarrow A(15) \rightarrow B(7) \rightarrow A(15) \rightarrow A(7) \rightarrow C(14) \rightarrow D(15) \rightarrow D(1)$$

Context-switch time is ignored in the lecture solution.

# Processes 2: Scheduling – Core Revision Summary

## 5. Comparison & Common Confusions

| Concept A        | Concept B           | Key difference / revision point                                                                                        |
|------------------|---------------------|------------------------------------------------------------------------------------------------------------------------|
| Ready queue      | Event queue         | Ready = waiting for CPU only; Event = blocked waiting for I/O/event. Event completion moves the process back to ready. |
| Time-out         | Program failure     | Time-out here = quantum expired, running → ready; it is normal preemption, not an application error.                   |
| Non-preemptive   | Preemptive          | Non-preemptive: only voluntary release; preemptive: timer/OS may forcibly interrupt.                                   |
| Blocking / yield | Preemption          | Blocking or <code>yield()</code> is voluntary; timer expiry is forced preemption.                                      |
| Response time    | Turnaround time     | Response ends at first execution; turnaround ends at final completion.                                                 |
| FCFS             | Round Robin         | Both retain queue order; RR adds periodic preemption/time slices. Very large quantum makes RR approach FCFS.           |
| SJF starvation   | Priority starvation | SJF can indefinitely delay long jobs; priority scheduling can indefinitely delay low-priority jobs.                    |
| CPU-bound        | I/O-bound           | CPU-bound tends to use full quanta; I/O-bound frequently blocks early and later rejoins the ready queue.               |

## 6. Exam / Revision Focus

### Must Know

- Definitions: scheduler/dispatcher, ready queue, event queue, time-out, preemptive/non-preemptive, context switch, response time, turnaround time, throughput, fairness, starvation.
- Scheduler horizons: long-term = admission, medium-term = swapping, short-term = next ready process.
- Algorithm identities: FCFS = non-preemptive FIFO; SJF = non-preemptive shortest estimate first; RR = preemptive FCFS with time quantum; priority = high-to-low, RR within equal priorities.

### Must Understand

- ♦ Why preemption prevents CPU monopolisation but adds overhead. ► ♦ Why an I/O-bound process can voluntarily leave the CPU even in a non-preemptive system. ► Why SJF minimises average turnaround time under the given assumptions. ► Why RR quantum size trades response time against throughput/context-switch cost. ► Why SJF and static highest-priority-first can cause starvation, while FCFS and ordinary RR normally do not.

### Lecture / dialogue exam questions

- **Where do schedulers fit in state transitions?** Long-term: new → ready; short-term: ready → running and running → ready decisions; medium-term: swap/suspend-related control.
- **Why can RR favour CPU-bound over I/O-bound processes?** CPU-bound jobs repeatedly use complete quanta; I/O-bound jobs often block early, lose their place, and must rejoin the ready queue after I/O. Priority/multiple queues can mitigate this.
- **Which algorithms can lead to starvation? SJF and highest-priority-first.** Long jobs can be continually overtaken by new short jobs; low-priority jobs can be continually overtaken by higher-priority jobs. FCFS/RR normally guarantee eventual turns for continuously ready jobs.
- **Does non-preemptive SJF mean the shortest process runs until completion? No.** It cannot be forcibly preempted, but it may block for I/O, terminate, or voluntarily call `yield()`.
- **How much harm can one user process do on a non-preemptive OS?** It can monopolise the CPU if it never blocks, terminates or yields, potentially preventing other processes from running indefinitely.

**Fast memory hook:** Ready = wait CPU; Event = wait event. Preemption = OS can force you off CPU. RR = FIFO + timer. Starvation risk = SJF + static priority.